

## Question 7

**Q:** A grayscale image is converted into a digital image. Describe the role of quantization in this process and its impact on image detail.

### Theoretical Concept: Role of Quantization

**Quantization** is the process of mapping continuous intensity values (voltages from the sensor) into a discrete set of digital values (e.g., 0 to 255). It determines the **color depth** or intensity resolution of the image.

**Impact:** If quantization levels are too low (e.g., 2-bit or 4-bit), smooth gradients become blocky, creating "false contours" or banding artifacts. High quantization preserves fine texture details.

### OpenCV Python Solution

[Copy](#)

```
import cv2 # Import OpenCV for image processing algorithms
import numpy as np # Import Numpy for matrix operations
import matplotlib.pyplot as plt # Import Matplotlib to plot images

# Step 1: Create an image with complex, smooth lighting
img = np.zeros((100, 100), dtype=np.uint8) # Create a blank black image array
cv2.circle(img, (50, 50), 50, 255, -1) # Draw a filled or outlined circle on the image
img = cv2.GaussianBlur(img, (49, 49), 20) # Soft blur gradient

# Step 2: Reduce quantization from 256 levels down to just 4 levels (2-bit)
# This forces every pixel to snap to one of 4 rigid gray values
quantized_4_level = np.floor(img / 64) * 64 # Round down each pixel value to quantized levels

# Step 3: Display the impact of poor quantization
plt.figure(figsize=(8,4)) # Create a new figure window for display
plt.subplot(121), plt.imshow(img, cmap='gray'), plt.title('256-Level Quantization')
```

```
plt.subplot(122), plt.imshow(quantized_4_level, cmap='gray'), plt.title('4-Level Qu  
plt.show() # Show the final plot window to the user
```

## Expected Output

The plot compares a perfectly smooth glowing sphere with a heavily distorted sphere made of concentric rigid rings. This clearly visualizes the physical impact of low quantization limits.

## How to Execute & Submit

### 1. Google Colab Execution:

- Open [Google Colab](#) and create a New Notebook.
- Click the  **Copy** button on the code block above.
- Paste it into a Colab cell and press **Shift + Enter** to run.

### 2. Uploading to GitHub:

- In Colab, go to **File > Save a copy in GitHub**.
- Authorize your GitHub account.
- Select your Computer Vision Lab repository and click **OK** to commit the code.